Control Flow Graphs and IR

CS 1622

Jarrett Billingsley

Class Announcements

- today: a second definition for **CFG** in the *same course*
  - naming is hard, ok
  - at least IR is new ;O

We have a problem

Our compiler is producing terrible code. Oh no!

- this is what my code generator produces for this tiny function.

```
fn main() {
  println_i(5);
}
```



```
main:
  sw   fp, -4(sp)
  sw   ra, -8(sp)
  move fp, sp
  addi sp, sp, -12
  sw   s0, -12(fp)
  addi sp, sp, -4
  li   s0, 5
  sw   s0, 0(sp)
  jal  println_i
  lw   s0, -12(fp)
  lw   ra, -8(fp)
  lw   fp, -4(fp)
  addi sp, sp, 12
  jr   ra
```

it's *a little* excessive. a better compiler might produce this (using the *real* MIPS ABI):

```
main:
  addi sp, sp, -4
  sw   ra, 0(sp)
  li   a0, 5
  jal  println_i
  lw   ra, 0(sp)
  addi sp, sp, 4
  jr   ra
```

so... how do we get closer to that?

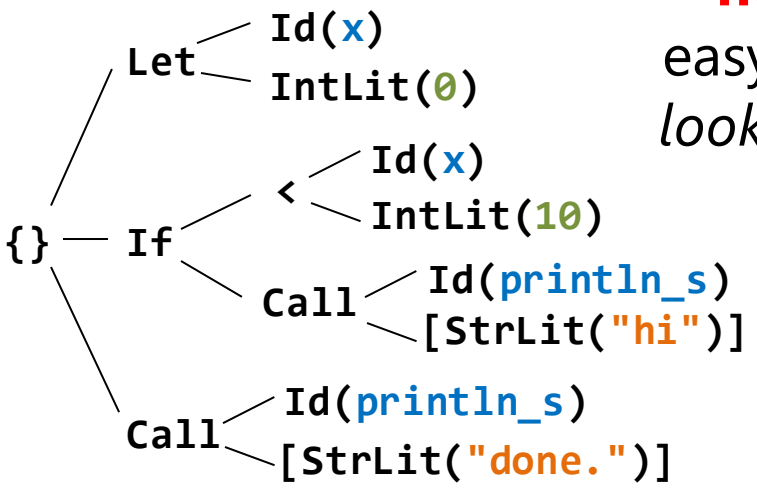
Something is holding us back

- there's a limit on the quality of code we can produce by going directly from the AST to machine/assembly code.
  - we could tweak and bodge and kludge improvements on what we have, and we'll **asymptotically** approach "good code" without ever really getting there.
- generating good code requires certain kinds of analyses to extract **more information** about the program being compiled.
  - **control flow analysis** determines the sequence in which pieces of code are run (i.e. "this runs before that; this *may* run before that").
  - **data flow analysis** determines how *values* are computed, copied, moved, reused, saved in variables, passed to functions, and so on.
- unfortunately, the AST is **not a good fit** for doing these analyses.

What's the S in AST stand for, again?

- consider these pieces of code and their ASTs.

```
let x = 0;
if x < 10 {
  println_s("hi!");
}
println_s("done.");
```

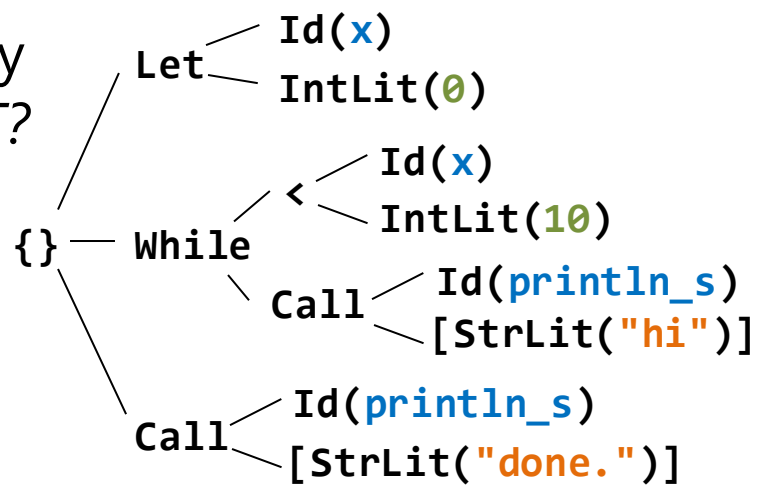


do these two pieces of code *behave* similarly?

not at all. is it easy to tell that by *looking at the AST?*

not really...

```
let x = 0;
while x < 10 {
  println_s("hi!");
}
println_s("done.");
```



Sufficiently Smart Compilers

- there's a (possible) mistake in this code that we humans can see easily.

```
let x = 0;
while x < 10 {
  println_s("hi!");
}
println_s("done.");
```

this loop **never terminates,** does it?

think about *how* you determined that.

1. **x** is assigned a **constant 0.**
2. **x** is *never assigned anywhere else,* not even in the loop.
3. so, **x** can *only* be **0** in the loop condition...
4. and **0 < 10**, meaning the condition is **tautologically true.**

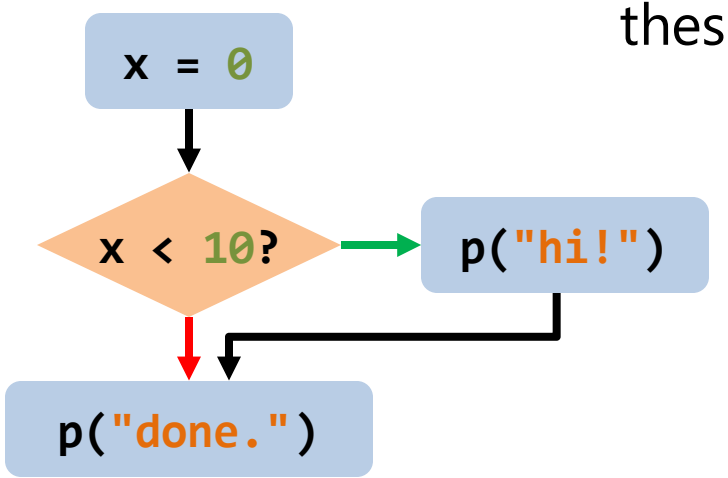
just looking at the AST, how would you prove these things?

what if the control flow were a lot more complex?

Program *execution* is not a tree.

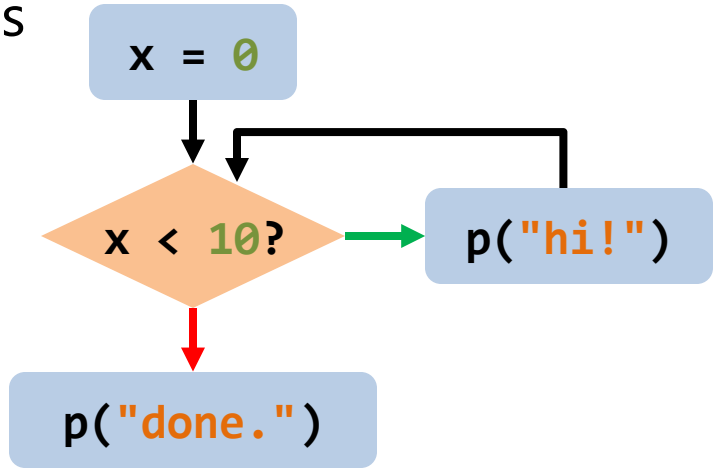
- the AST models ***what the programmer typed.*** there's another way of representing programs that models **how the program executes.**

```
let x = 0;
if x < 10 {
  println_s("hi!");
}
println_s("done.");
```



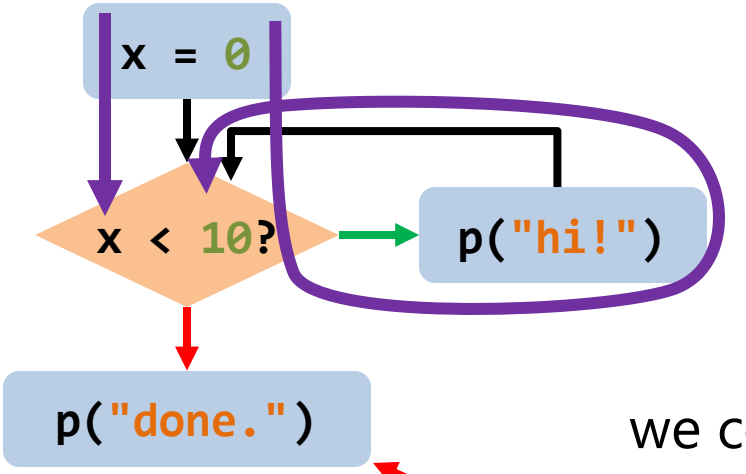
flowcharts make it immediately obvious that the **structure** of these two programs is different.

```
let x = 0;
while x < 10 {
  println_s("hi!");
}
println_s("done.");
```



Sufficiently Simple Algorithms

- a flowchart makes it easier to detect the problem here.



if we consider **all possible paths** that take us to the condition…

then we can see that the **only possible value** for **x** is the constant `0`.

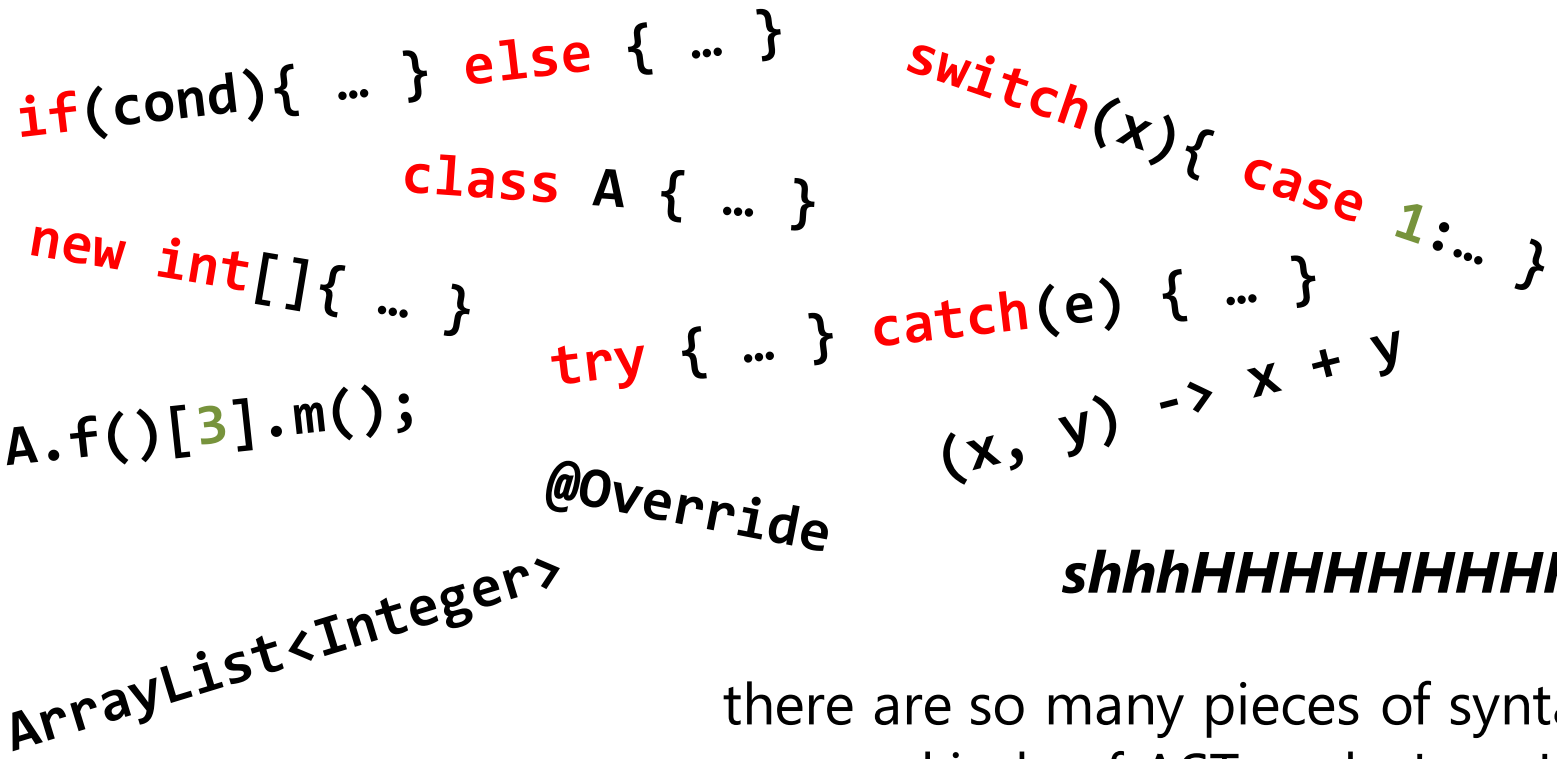
we could give an error (or warning) about the condition, or tell them that this last piece of code is **unreachable.**

this sort of algorithm is crucial for detecting subtle bugs, doing certain kinds of semantic analysis, and generating better code.

Intermediate Representation (IR)

Real languages are a lot bigger.

- the **AST** is an abstract representation of the *source* language.
- but the source language can be… complicated.



there are so many pieces of syntax! so many kinds of AST nodes! aaaHHH

Tight coupling is bad.

- a codegen algorithm that goes directly from source AST to target machine code is also **not very flexible.**



what if you want to modify the source language? or change the AST representation?

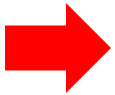
what if you want to output a different target language?

this algorithm **tightly couples** the input and output: if you want to change either of them, you have to change the algorithm.

Hidden complexity

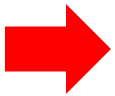
- the AST can also **hide operations and control flow** that we have to know about to be able to analyze the code properly.

```
if cond { code }
```



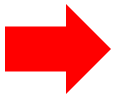
```
if !<cond> goto end
<code>
end:
```

```
while cond { code }
```



```
top: if !<cond> goto end
<code>
goto top
end:
```

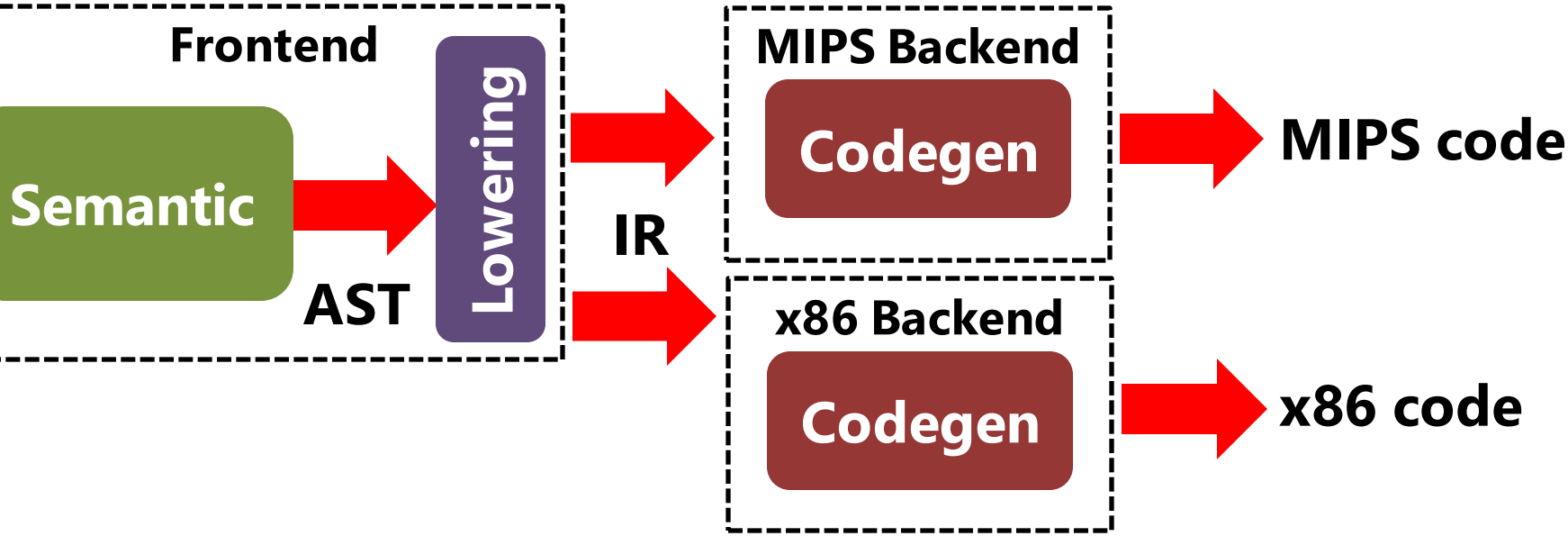
```
for i in lo, hi { code }
```



```
i = <lo>
top: if i >= <hi> goto end
<code>
i += 1
goto top
end:
```

So instead…

- an **intermediate representation (IR)** is essentially a **third language** that acts as a bridge between the source and target languages.



now we can work on the frontend and backend parts **in isolation**, **swap out** the backend, **change the AST** without having to change the codegen, etc…

IR Goals

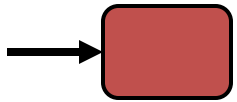
- we want a language with a **small number** of **simple** operations
  - ...which can be **efficiently implemented** on most/all targets
- so, something *similar to* assembly language...
  - but *without* being tied to a particular ISA.
- a representation amenable to **optimization** would also be nice...
  - optimization rewrites code to do the same thing, but faster.
  - simple code is easier to optimize, so we've got that covered.
- finally, we'll want to represent control flow as a **graph,**
  - as this enables lots of analyses for optimization and codegen that a linear or tree structure would not.
- the IR we'll be discussing is based on rustc's MIR,
  - but lots of IRs are similar in their goals and structure.

Our IR

The big picture

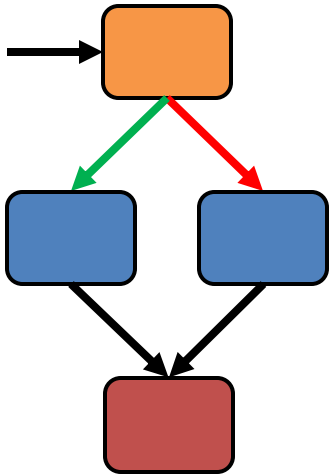
- each function in the source language will be represented by a **CFG**.
  - this is a **control flow graph:** essentially, a flowchart.

```
fn main() {
  println_i(5);
}
```



each node in the CFG is called a **basic block.** this function only has one.

```
fn test(x: bool) {
  if x {
    println_s("y");
  } else {
    println_s("n");
  }
}
```



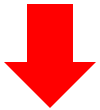
basic blocks can have 0, 1, or 2 **successors:** what will be run after the basic block completes.

blocks with 2 successors are clearly making decisions…

The little picture

- each basic block will contain **instructions** in a simple language.
  - basically, each line only does **one** thing.

```
d = (x + y) * (-z / w)
```



```
$t1 = x + y
$t2 = -z
$t3 = $t2 / w
d   = $t1 * $t3
```

destination, source, operation

if this reminds you of asm, good.

```
add  $t1, x, y
neg  $t2, z
div  $t3, $t2, w
mul  d,   $t1, $t3
```

it's kind of like asm **without loads/stores,** and with **infinite temporary registers.**

Locals/temporaries

- each function has a list of **locals:** places where values can be stored.
- some of these are **local variables** that the user declared; others are **temporaries** created by the code generation.

```
fn func(a: int, b: int) {
  let x = (a + b) * 3;
  println_i(x);
}
```

each local has a **type** and optionally a **name** (from the code).

Locals

Idx	Name	Type
0		()
1	a	int
2	b	int
3		int
4	x	int

Code

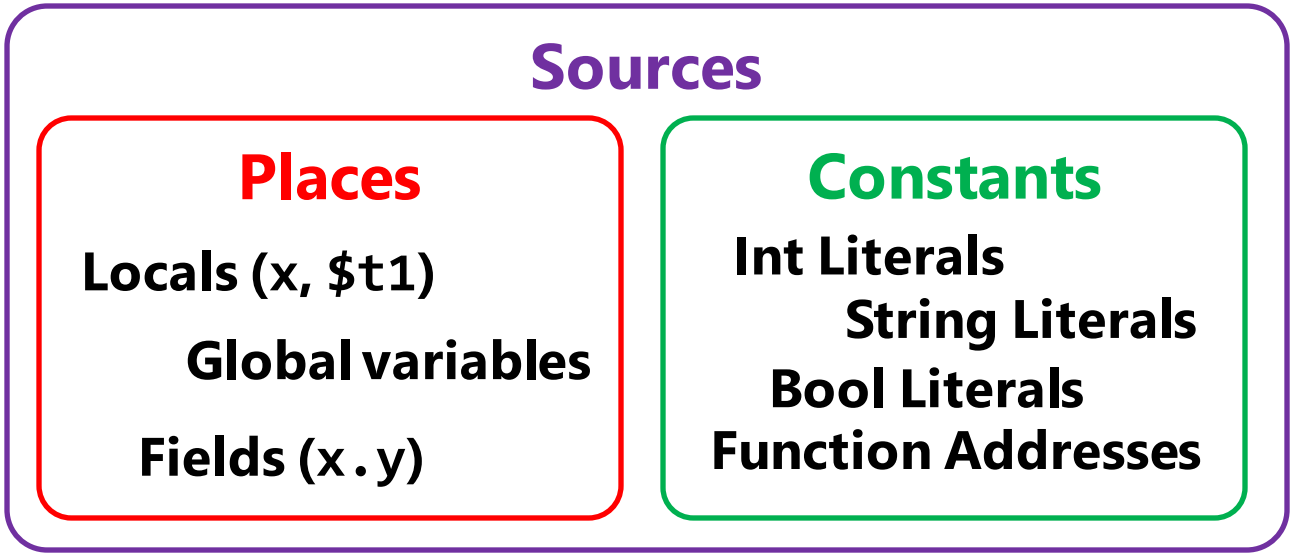
```
$t3 = a + b
x = $t3 * 3
println_i(x)
return
```

if a local has no name, it's referred to by its index in the code (like **$t3**).

we'll come back to **$t0** shortly...

Places, Constants, and Sources

- the operands of instructions are a little freer than in assembly.



a **place** is what can appear on the **LHS of an assignment.**

a **constant** is just what it sounds like. **function addresses** are used in function calls.

both places and constants can be used as **sources,** which are the operands on the RHS of assignments, arguments, etc.

Instruction examples

- here are some examples of valid instructions in this IR language.

`x = 1`	simple assignment of a constant to a local.
`$t5.y = x.y`	field access can happen on either (or both) side(s) of an assignment.
`x.y = a.b.c + 1`	this is still just one instruction. field access doesn't count as an "operation;" `+` does.
`$t0 = f(1, $t1)`	function calls can take any number of arguments, which are all sources.
`p = &println_s`	a function's address can be put in a place...
`p("hi")`	...and then that place can be called.

Terminators

- a basic block (BB) contains 0 or more regular instructions, and ends with a **terminator,** which is a **control flow** instruction.

a **return** terminator returns from the function.

```
...
return
```

a **goto** terminator **unconditionally** goes to another BB.

```
...
goto bb7
```

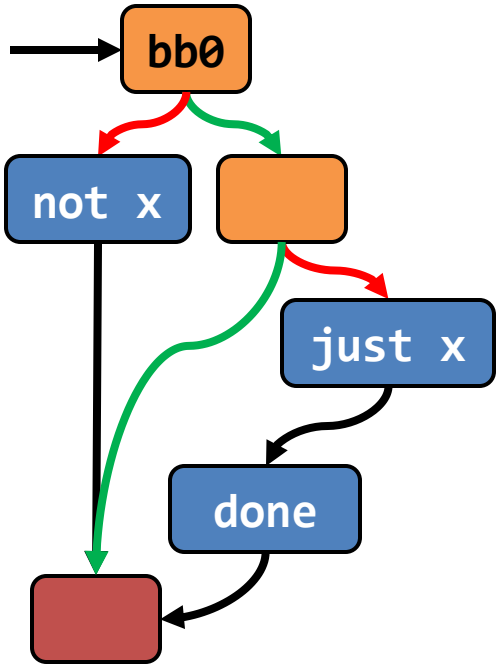
and a **conditional** terminator goes to **one of two BBs,** based on the condition.

```
...
if $t4 bb9 else bb12
```

One way in, one way out

- every function has at least one BB, its **entry point.** it's named `bb0`.
- we will also ensure every function has ***exactly* one BB with a return.**
  - all `return` statements will jump (`goto`) to it.

```
fn f(x: bool, y: bool) {
  if x {
    if y {
      return;
    }
    println_s("just x");
  } else {
    println_s("not x");
    return;
  }
  println_s("done");
}
```



notice how both `return`**s** become arrows to the red BB (which has a return terminator).

this is going to make certain analyses much easier, since some of them start at the *end* and go backwards.

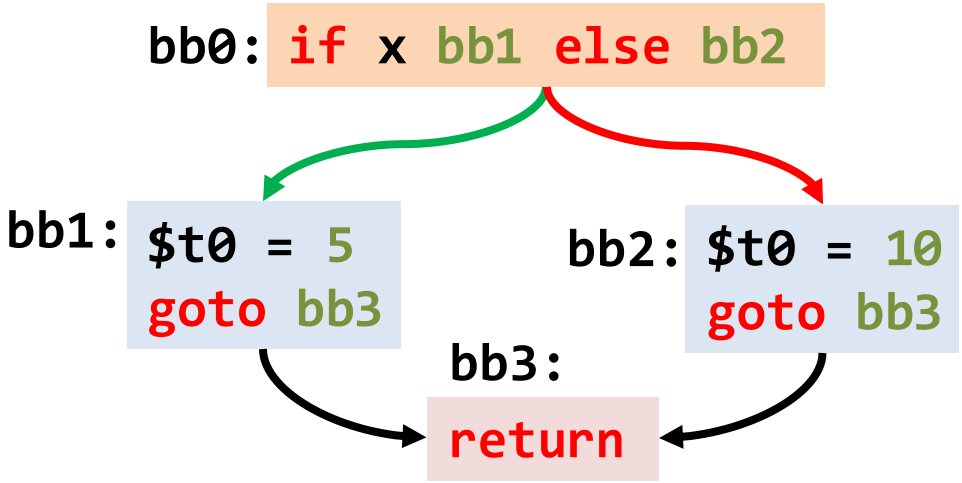
Return values

- there is a special temporary, **`$t0`**, which is used for the return value.
- similar to the **`$v0`** register in MIPS, a value is returned by assigning something into it before you get to the **`return`** terminator.

```
fn ret(x: bool): int {
  if x {
    return 5;
  } else {
    return 10;
  }
}
```

Locals

Idx	Name	Type
0		int
1	x	bool



the special parsing rule about **returning statements** (remember that?) along with the typechecking pass will ensure that this location is always assigned a value.

Codegen, but different

Lowering

- **lowering** is the process of converting the high-level, *abstract* syntax tree into the mid-level IR that we just looked at.
- it's a kind of code generation, so there are some similarities with what we talked about before. but in many ways it's a lot simpler:
  - there are **infinite** temporary locations, so there's no need for complex register allocation algorithms.
  - it also means there's no need for **stack frames, loads, stores etc.**
  - all locations are **typed,** so some implementation details are left out (e.g. **a + b** can mean addition or string concatenation, like in the source language)
- so all we have to do is:
  - convert complex operations into sequences of simpler ones
  - build the CFG according to the control flow statements

Building the CFG: straight-line code (animated)

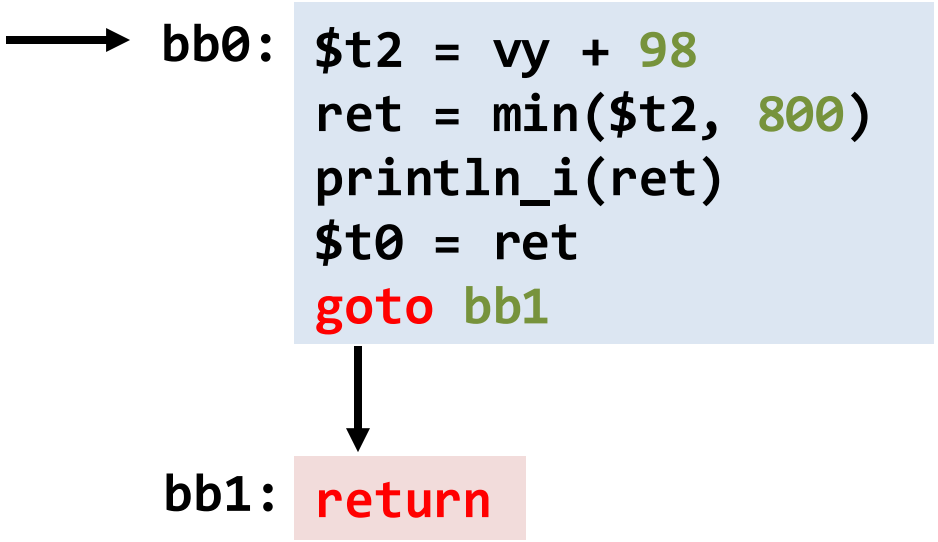
- if a function has no control flow, it's super simple.

```
fn do_gravity(vy: int): int
  let ret = min(vy + 98, 800);
  println_i(ret);
  return ret;
}
```

Locals

Idx	Name	Type
0		int
1	vy	int
2		int
3	ret	int

you start with an empty BB, and generate IR instructions into it.

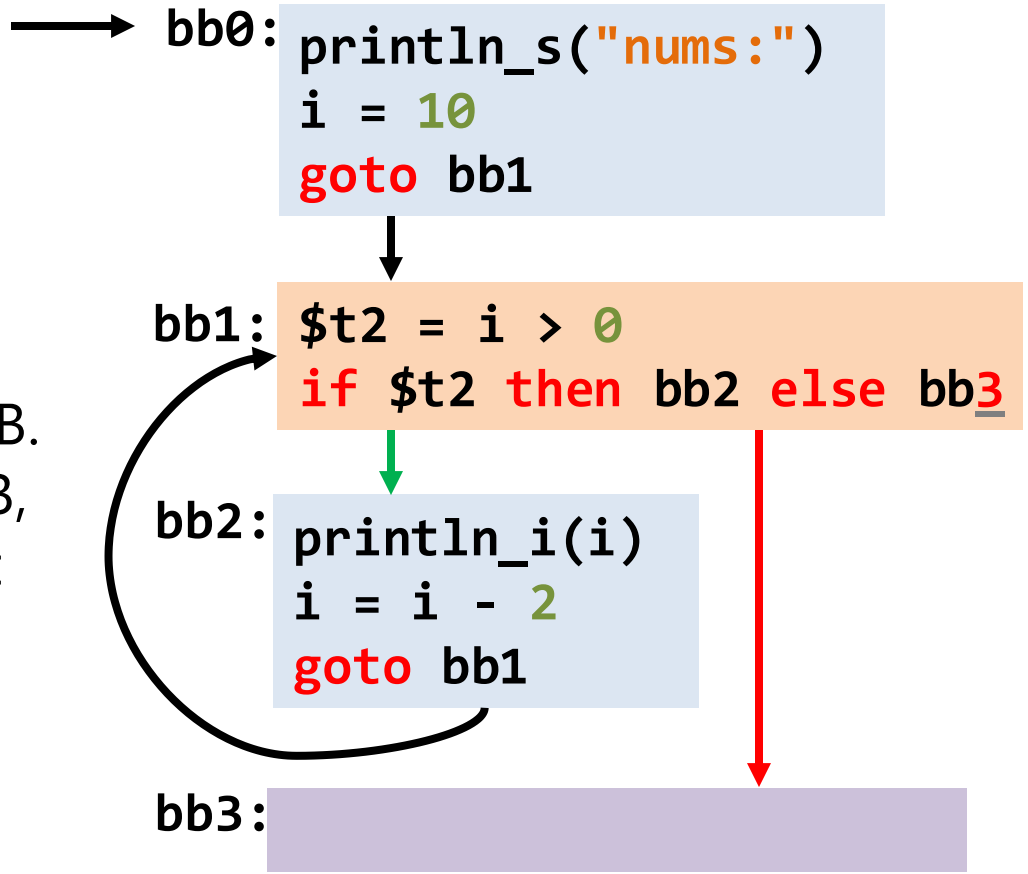


we can make a new BB for the **return** like here, or special-case it if we want a simpler CFG.

Building the CFG: a while loop (animated)

- control flow structures tell us exactly where BBs need to begin/end.

```
println_s("nums:");
let i = 10;
while i > 0 {
  println_i(i);
  i = i - 2;
}
```

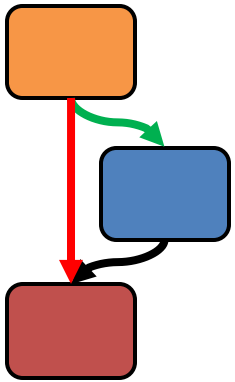


the loop needs a new BB.
when we start a new BB,
the previous one must
goto the new one.

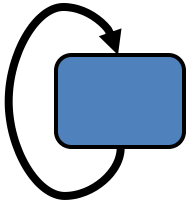
done with the body;
make a new BB after it.

The shape of control flow

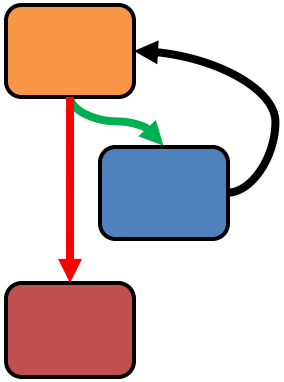
- what kind of control flow statement would make each CFG?



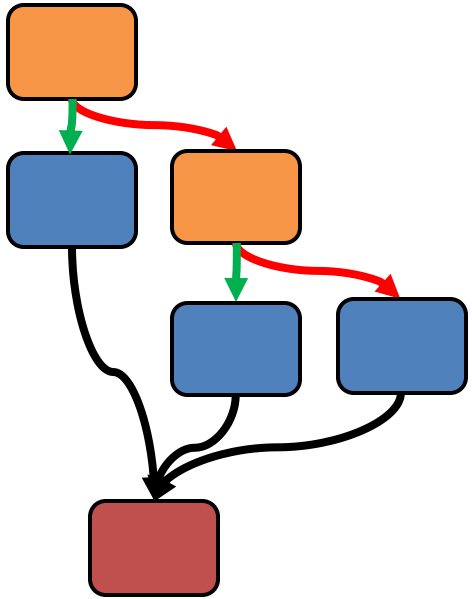
if



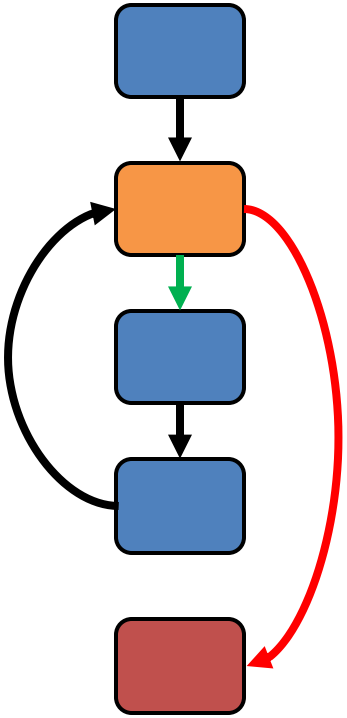
while true



while



if … else if … else



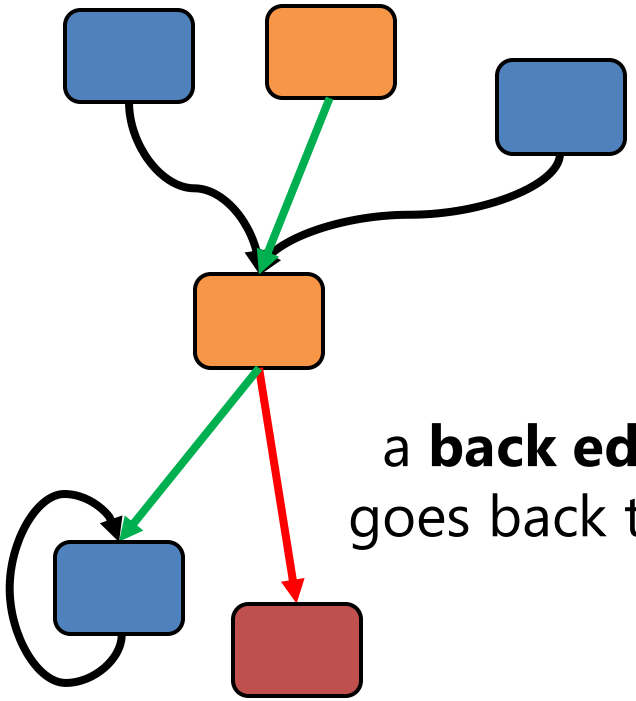
for(i=a; i<b; i++){…}

Some graph-related stuff

How they connect (animated)

- graphs always have all sorts of vocabulary, huh?

a BB's **predecessors** are all the BBs that **point to it.**



a BB's **successors** are the BBs which it **points to.**

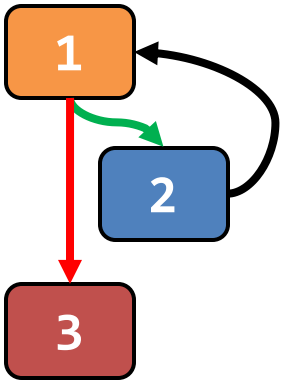
a BB can be its own successor, in which case it is *also* its own predecessor!

a **back edge** is one that goes back to an earlier BB.

remember: **whenever you see cyclic graphs, *things are gonna get weird.***

Visiting arbitrary graphs

- visiting a **tree** is easy: just recurse for each child node.
- but doing that for an *arbitrary* graph will get you into trouble.




```
fn visit_node(n) {
  print(n's number);
  for s in n.successors() {
    visit_node(s);
  }
}
```

if we run this code on this graph, **what happens?**

we'll visit node 1… then 2… then 1… **then 2… then 1…**

clearly this isn't sufficient. **when visiting an arbitrary graph,** you **must manually keep track of which nodes have already been visited.**

Postorder using depth-first traversal

- a really common way to visit CFGs is using **postorder:** each node is visited **after all its successors have been visited.**

```
fn visit_node(n, visited) {
  if visited[n] { return; }
  visited[n] = true;
  for s in n.successors() {
    visit_node(s, visited);
  }
  actually "visit" n here!
}
```

this bit is what prevents us from looping infinitely.

and this is where we do whatever work is needed to "visit" **n**, *after* we've visited its successors.

if we say, printed out the node's name/number there, this would print out the postorder of the graph.